KAFKA CORPORATE TRAINING

# HA/DR, Disaster Recovery & Real-World Scenarios

*Day 5 — Production Resilience, Capacity Planning & Failure Analysis*

Rajesh Pasham · Day 5 · Final Day

# Day 5 — Agenda

**01** **High Availability**
*Replication Factors · ISR Management · Preferred Leader Election*

**02** **MirrorMaker 2 / DR**
*Active-Passive · Active-Active · Offset Translation · Consumer Failover*

**03** **Backup & Restore**
*Log Segment Backup · Schema Export · Config Versioning*

**04** **Upgrade & Patching**
*Rolling Upgrades · Protocol Versions · ZooKeeper → KRaft Migration*

**05** **Capacity Planning**
*Partitions · Retention · Storage Formula · Network & CPU*

**06** **Real-World Scenarios**
*Message Loss · Duplicates · Broker Failover · DC Failover*

MODULE 01

# High Availability

*Replication · ISR · Preferred Leader Election*

# Replication Factor — Foundation of HA

- RF = number of copies of each partition across the cluster
- RF=1: No redundancy — dev/test only. Single broker failure = data unavailable.
- RF=2: Survives 1 failure. Minimal overhead. Low-criticality topics.
- RF=3: Industry production standard. Survives 1 failure + 1 in degraded mode.
- Rule: Maintain at least RF+1 brokers (e.g. RF=3 needs ≥4 brokers for safe rolling restarts)
- Set default: default.replication.factor=3 in server.properties
- Override per-topic using --replication-factor flag at create time
- RF cannot exceed broker count — plan broker count first

# ISR + min.insync.replicas — The Safety Net

## ISR (In-Sync Replicas)

- Runtime list of replicas caught up with leader
- Only ISR members can become leader on failover
- Shrinks when follower lags > replica.lag.time.max.ms
- Expands when follower catches back up
- Monitor: UnderReplicatedPartitions JMX metric
- Shrunk ISR + unclean election = data loss risk

## min.insync.replicas Config

- Min ISR required to accept writes (acks=all)
- Recommended: min.isr=2 with RF=3
- If ISR < min.isr → NotEnoughReplicasException
- Trade-off: availability vs durability
- Set: kafka-configs.sh --alter
- --add-config min.insync.replicas=2

# unclean.leader.election — Critical HA Setting

Controls whether a non-ISR replica can become leader:

```
# SAFE (production default) — prevents data loss
unclean.leader.election.enable=false
# Behavior: partition stays OFFLINE until ISR member available
# Durability: NO data loss
# Availability: may have downtime

# UNSAFE — risks data loss
unclean.leader.election.enable=true
# Behavior: out-of-sync replica becomes leader immediately
# Durability: POSSIBLE DATA LOSS (missed writes)
# Availability: no downtime

# Recommended: false for financial/critical data
# Only consider true for high-availability logging pipelines
```

MODULE 02

# MirrorMaker 2 & Disaster Recovery

*Cross-cluster replication for zero-data-loss DR*

# MirrorMaker 2 Architecture

### MirrorSourceConnector

Replicates topic data from source to target. Prefixes topic names with source cluster alias (e.g. primary.orders).

### MirrorCheckpointConnector

Mirrors committed consumer group offsets. Enables consumers to resume on DR cluster at the right position after failover.

### MirrorHeartbeatConnector

Produces heartbeat messages every second. Measures replication latency and verifies the pipeline is alive.

### Offset Translation

Source offsets ≠ DR offsets. MM2's MirrorClient API translates offsets automatically for seamless consumer failover.

# MirrorMaker 2 — mm2.properties Config

Full configuration for active-passive DR setup:

```
clusters = primary, dr
primary.bootstrap.servers = primary1:9092,primary2:9092
dr.bootstrap.servers      = dr1:9092,dr2:9092

# Enable replication direction
primary->dr.enabled = true
primary->dr.topics  = .*
primary->dr.groups  = .*

# Consumer group offset sync (key for low-RPO failover)
primary->dr.sync.group.offsets.enabled          = true
primary->dr.sync.group.offsets.interval.seconds = 60

# Replication factor for MM2 internal topics on DR
replication.factor                        = 3
checkpoints.topic.replication.factor      = 3
```

# Active-Passive vs Active-Active DR

| Aspect | Active-Passive | Active-Active |
|---|---|---|
| Description | Primary handles all traffic. DR is hot standby. | Both clusters serve traffic simultaneously. |
| Replication | Unidirectional: primary → DR | Bidirectional: primary ↔ DR |
| RTO | Minutes (manual) to seconds (auto) | Near-zero — instant reroute |
| RPO | Near-zero with sync.group.offsets | Near-zero |
| Complexity | Low — simple topology, easier operations | High — needs loop prevention logic |
| Use Case | Most enterprises, simpler to manage | Global, latency-sensitive applications |

MODULE 03

# Backup & Restore Strategy

*Protecting Kafka data, configs, and schemas*

# What Needs Backing Up?

## Topic Log Segments

Files in log.dirs. Back up with OS-level tools (tar, rsync, LVM snapshot). Stop broker first for consistency.

## Consumer Offsets

__consumer_offsets topic. Mirror via MirrorMaker 2 or use kafka-consumer-groups.sh to export offsets.

## Configs & Schemas

Topic/broker configs → kafka-configs.sh --describe. Schema Registry → REST API export. Store in Git (IaC).

## ZooKeeper / KRaft

ZK dataDir snapshot or __cluster_metadata topic for KRaft. Always included in broker backup procedure.

# Backup & Restore Commands

Key commands for log segment backup, verification, and restore:

```
# 1. Graceful broker shutdown (consistent state)
kafka-server-stop.sh

# 2. Backup log directory
tar -czf /backup/kafka-$(date +%Y%m%d).tar.gz /data/kafka-logs/
aws s3 cp /backup/kafka-20260315.tar.gz s3://my-kafka-backups/

# 3. Verify a log segment
kafka-dump-log.sh --files /data/kafka-logs/payments-0/*.log \
  --print-data-log | head -20

# 4. Restore
tar -xzf /backup/kafka-20260315.tar.gz -C /
kafka-server-start.sh config/server.properties &
```

MODULE 04

# Upgrade & Patching Strategy

*Zero-downtime rolling upgrades and ZooKeeper → KRaft migration*

# Rolling Upgrade — Step by Step

- Step 1: Review release notes. Test upgrade in staging environment first.
- Step 2: Pin inter.broker.protocol.version to CURRENT version on ALL brokers
- Step 3: Stop broker 1 → install new Kafka binary → start broker 1
- Step 4: Wait for broker 1 to fully rejoin — monitor UnderReplicatedPartitions = 0
- Step 5: Repeat for broker 2, broker 3, ... broker N (one at a time)
- Step 6: After ALL brokers upgraded → update inter.broker.protocol.version to NEW version
- Step 7: Update log.message.format.version to NEW version
- Step 8: Rolling restart all brokers to apply new protocol. Verify cluster health.

# Protocol Version Management

## During Upgrade Window

- inter.broker.protocol.version=OLD
- log.message.format.version=OLD
- → Mixed versions allowed
- → Old + new brokers can coexist
- → New features NOT yet active
- Brokers negotiate lowest protocol
- Safe to restart brokers one by one

## After Full Upgrade

- inter.broker.protocol.version=NEW
- log.message.format.version=NEW
- → Rolling restart to activate
- → New message format features on
- → Old clients may need updating
- ⚠️ Do NOT update before ALL brokers
- are on the new version

MODULE 05

# Capacity Planning

*Partitions · Retention · Storage · Network · CPU*

# Key Capacity Planning Numbers

**RF × data**

**Storage Multiplier**

Every byte stored RF times across the cluster

**~10 MB/s**

**Per-Partition Producer**

Typical max throughput per partition (SSD, no compression)

**≤ 50 MB/s**

**Per-Partition Consumer**

Consumers can read faster; network is usually the limit

**+25–30%**

**Storage Headroom Buffer**

Add to raw calculation for indexes, compaction, overhead

# Storage Capacity Formula

Calculate total storage needed for a topic:

```
# Formula
Storage = rate(msg/s) × size(bytes) × retention(s) × RF

# Example: Payments topic
rate      = 500 msg/s
size      = 2 KB  =  2,048 bytes
retention = 7 days = 604,800 seconds
RF        = 3

raw_per_partition = 500 × 2048 × 604800 = 619 GB
total_cluster     = 619 GB × 3          = 1.86 TB
with_buffer       = 1.86 TB × 1.25      = 2.33 TB

# Partitions formula
partitions = max(target_MB_s / 10,  target_MB_s / 50)
```

# Retention & Compaction Strategies

## Time / Size Retention

- cleanup.policy=delete (default)
- log.retention.ms=604800000 (7d)
- log.retention.bytes=107374182400
- → Delete old segments when threshold hit
- Use for: event streams, logs, metrics
- Segment size: log.segment.bytes=1GB
- Smaller segments = faster retention cleanup

## Log Compaction

- cleanup.policy=compact
- Retains latest value per key
- Ideal for: CDC, config topics, state
- min.compaction.lag.ms — delay before eligible
- min.cleanable.dirty.ratio — trigger threshold
- Combine: cleanup.policy=delete,compact
- → Compaction + time-based retention

MODULE 06

# Real-World Failure Scenarios

*Message Loss · Duplicates · Failovers*

# Message Loss — Root Causes & Fixes

| Root Cause | Detection | Fix |
|---|---|---|
| acks=0/1 | Producer error rate = 0 but messages missing on consumer | acks=all + enable.idempotence=true |
| unclean leader election | OfflinePartitions = 0 but offset gap after leader change | unclean.leader.election.enable=false |
| ISR drops + min.isr=1 | ISR < RF, no alert, writes succeed with only 1 copy | min.insync.replicas=2 with RF=3 |
| Consumer auto-commit early | Messages processed twice on restart or offset jumps ahead | enable.auto.commit=false — manual commit |

# Message Duplication — Root Causes & Fixes

| Scope | Root Cause | Solution |
|---|---|---|
| Producer | Retry on timeout — broker wrote but ack lost | enable.idempotence=true (per-partition EOS) |
| Producer | Cross-partition / cross-session duplicates | Kafka Transactions (transactional.id) |
| Consumer | Crashes after processing but before offset commit | Manual commit after processing completes |
| Consumer | Partition reassigned mid-processing (rebalance) | Cooperative rebalance + idempotent consumer logic |
| End-to-End | Exactly-once across read-process-write | Kafka Streams or isolation.level=read_committed |

# Kafka Transactions — Exactly-Once

Producer + consumer config for full exactly-once semantics:

```
// Producer: enable transactions
props.put("transactional.id",   "payments-producer-1");
props.put("enable.idempotence",  "true");
props.put("acks",                "all");

producer.initTransactions();
try {
  producer.beginTransaction();
  producer.send(new ProducerRecord<>("payments", key, value));
  producer.commitTransaction();
} catch (Exception e) {
  producer.abortTransaction();
}

// Consumer: read only committed messages
props.put("isolation.level", "read_committed");
```

# Failover Scenarios — Broker & Controller

- Single broker crash → Controller detects → leader election for affected partitions
- New leaders elected from ISR — typically < 5 seconds on healthy cluster
- Producers / consumers auto-reconnect using bootstrap.servers list
- Monitor: OfflinePartitionsCount returns to 0 → recovery complete
- Controller failure: ZK mode — first broker to claim /controller wins
- KRaft mode — Raft election among controller nodes, faster and more reliable
- Preferred leader election rebalances leaders after recovery (or use auto.leader.rebalance.enable=true)
- Always monitor UnderReplicatedPartitions during and after any broker event

# Full Datacenter Failover — Runbook

- T+0: Alerting fires — primary cluster unreachable (broker health checks failing)
- T+2: On-call engineer confirms outage. DR readiness check: replication lag? data currency?
- T+5: Verify DR cluster health — all brokers up, all partitions with leaders
- T+8: Execute DNS/LB cutover to DR cluster bootstrap.servers
- T+9: Redirect consumer groups — use MirrorMaker offset translation for correct resume
- T+10: First messages confirmed flowing on DR cluster — service restored
- T+15: Stakeholder communication. Incident bridge open. RTO/RPO metrics captured.
- T+60+: Root cause investigation on primary. Plan for failback when ready.

# Training Complete!

*5-Day Kafka Corporate Training*

- Day 1: Kafka Fundamentals & Architecture
- Day 2: Administration & Operations
- Day 3: Monitoring, Troubleshooting & Diagnostics
- Day 4: Advanced Kafka Operations (Level 3)
- Day 5: HA/DR, Disaster Recovery & Real-World Scenarios ✓